

模型建立和评估

2026 年 5 月 22 日

1 模型建立和评估

经过前面的探索性数据分析我们可以很清楚的了解到数据集的情况

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from IPython.display import Image
```

```
[2]: %matplotlib inline
```

```
[3]: plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
plt.rcParams['figure.figsize'] = (10, 6) # 设置输出图片大小
```

```
[15]: # 读取训练数据集
train = pd.read_csv('/home/jovyan/work/datasets/
↳ 6645a647c4db0a94029f0bf5-momodel/dataset/train.csv')
train.shape
```

```
[15]: (891, 12)
```

```
[16]: train.head()
```

```
[16]:
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	

3	4	1	1
4	5	0	3

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

1.1 1. 特征工程

任务一：缺失值填充

* 对分类变量缺失值：填充某个缺失值字符 (NA)、用最多类别的进行填充 * 对连续变量缺失值：填充均值、中位数、众数

```
[18]: # 对分类变量进行填充
train['Cabin'] = train['Cabin'].fillna('NA')
train['Embarked'] = train['Embarked'].fillna('S')
```

```
[19]: # 对连续变量进行填充
train['Age'] = train['Age'].fillna(train['Age'].mean())
```

```
[20]: # 检查缺失值比例
train.isnull().sum().sort_values(ascending=False)
```

```
[20]: PassengerId    0
      Survived      0
      Pclass       0
      Name         0
```

```

Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
dtype: int64

```

任务二：编码分类变量

```

[21]: # 取出所有的输入特征
data = train[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']]

```

```

[22]: # 进行虚拟变量转换
data = pd.get_dummies(data)

```

```

[23]: data.head()

```

```

[23]:   Pclass   Age  SibSp  Parch    Fare  Sex_female  Sex_male  Embarked_C  \
0        3  22.0     1     0   7.2500           0         1           0
1        1  38.0     1     0  71.2833           1         0           1
2        3  26.0     0     0   7.9250           1         0           0
3        1  35.0     1     0  53.1000           1         0           0
4        3  35.0     0     0   8.0500           0         1           0

```

```

      Embarked_Q  Embarked_S
0              0           1
1              0           0
2              0           1
3              0           1
4              0           1

```

1.2 2. 模型搭建

- 处理完前面的数据我们就得到建模数据，下一步是选择合适模型

- 在进行模型选择之前我们需要先知道数据集最终是进行**监督学习**还是**无监督学习**
- 除了根据我们任务来选择模型外，还可以根据数据样本量以及特征的稀疏性来决定
- 刚开始我们总是先尝试使用一个基本的模型来作为其 baseline，进而再训练其他模型做对比，最终选择泛化能力或性能比较好的模型

思考：数据集哪些差异会导致模型在拟合数据是发生变化

算法选择路径图如下：

任务一：切割训练集和测试集 * 按比例切割训练集和测试集（一般测试集的比例有 30%、25%、20%、15% 和 10%） * 按目标变量分层进行等比切割 * 设置随机种子以便结果能复现

提示 1 * 切割数据集是为了后续能评估模型泛化能力 * sklearn 中切割数据集的方法为 `train_test_split` * 查看函数文档可以在 jupyter notebook 里面使用 `train_test_split` 后回车即可看到 * 分层和随机种子在参数里寻找

思考 1 * 什么情况下切割数据集的时候不用进行随机选取

```
[24]: from sklearn.model_selection import train_test_split
```

```
[25]: # 一般先取出 X 和 y 后再切割，有些情况会使用到未切割的，这时候 X 和 y 就可以用
X = data
y = train['Survived']
```

```
[26]: # 对数据集进行切割
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,
↪random_state=0)
```

```
[27]: # 查看数据形状
X_train.shape, X_test.shape
```

```
[27]: ((668, 10), (223, 10))
```

任务二：模型创建 * 创建基于线性模型的分类模型（逻辑回归） * 创建基于树的分类模型（决策树、随机森林） * 查看模型的参数，并更改参数值，观察模型变化

提示 2 * 逻辑回归不是回归模型而是分类模型，不要与 `LinearRegression` 混淆 * 随机森林其实是决策树集成为降低决策树过拟合的情况 * 线性模型所在的模块为 `sklearn.linear_model` * 树模型所在的模块为 `sklearn.ensemble`

思考 2 * 为什么线性模型可以进行分类任务，背后是怎么的数学关系 * 对于多分类问题，线性模型是怎么进行分类的

```
[29]: from sklearn.linear_model import LogisticRegression
      from sklearn.ensemble import RandomForestClassifier
```

```
[30]: # 默认参数逻辑回归模型
      lr = LogisticRegression()
      lr.fit(X_train, y_train)
```

```
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)

```
[30]: LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                        intercept_scaling=1, l1_ratio=None, max_iter=100,
                        multi_class='auto', n_jobs=None, penalty='l2',
                        random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                        warm_start=False)
```

```
[31]: # 查看训练集和测试集 score 值
      print("Training set score: {:.2f}".format(lr.score(X_train, y_train)))
      print("Testing set score: {:.2f}".format(lr.score(X_test, y_test)))
```

Training set score: 0.80

Testing set score: 0.78

```
[32]: # 调整参数后的逻辑回归模型
      lr2 = LogisticRegression(C=100)
      lr2.fit(X_train, y_train)
```

```
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
```

```
to converge (status=1):
```

```
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
```

```
[32]: LogisticRegression(C=100, class_weight=None, dual=False, fit_intercept=True,
                        intercept_scaling=1, l1_ratio=None, max_iter=100,
                        multi_class='auto', n_jobs=None, penalty='l2',
                        random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                        warm_start=False)
```

```
[33]: print("Training set score: {:.2f}".format(lr2.score(X_train, y_train)))
      print("Testing set score: {:.2f}".format(lr2.score(X_test, y_test)))
```

```
Training set score: 0.81
```

```
Testing set score: 0.78
```

```
[34]: # 默认参数的随机森林分类模型
      rfc = RandomForestClassifier()
      rfc.fit(X_train, y_train)
```

```
[34]: RandomForestClassifier(bootstrap=True, ccp_alpha=0.0, class_weight=None,
                        criterion='gini', max_depth=None, max_features='auto',
                        max_leaf_nodes=None, max_samples=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, n_estimators=100,
                        n_jobs=None, oob_score=False, random_state=None,
                        verbose=0, warm_start=False)
```

```
[35]: print("Training set score: {:.2f}".format(rfc.score(X_train, y_train)))
      print("Testing set score: {:.2f}".format(rfc.score(X_test, y_test)))
```

Training set score: 0.99

Testing set score: 0.80

[36]: # 调整参数后的随机森林分类模型

```
rfc2 = RandomForestClassifier(n_estimators=100, max_depth=5)
rfc2.fit(X_train, y_train)
```

```
[36]: RandomForestClassifier(bootstrap=True, ccp_alpha=0.0, class_weight=None,
                             criterion='gini', max_depth=5, max_features='auto',
                             max_leaf_nodes=None, max_samples=None,
                             min_impurity_decrease=0.0, min_impurity_split=None,
                             min_samples_leaf=1, min_samples_split=2,
                             min_weight_fraction_leaf=0.0, n_estimators=100,
                             n_jobs=None, oob_score=False, random_state=None,
                             verbose=0, warm_start=False)
```

```
[37]: print("Training set score: {:.2f}".format(rfc2.score(X_train, y_train)))
      print("Testing set score: {:.2f}".format(rfc2.score(X_test, y_test)))
```

Training set score: 0.86

Testing set score: 0.83

任务三：输出模型预测结果 * 输出模型预测分类标签 * 输出不通分类标签的预测概率

提示 3 * 一般监督模型在 sklearn 里面有个 predict 能输出预测标签，predict_proba 则可以输出标签概率

思考 3 * 预测标签的概率对我们有什么帮助

[38]: # 预测标签

```
pred = lr.predict(X_train)
```

[39]: # 此时我们可以看到 0 和 1 的数组

```
pred[:10]
```

```
[39]: array([0, 1, 1, 1, 0, 0, 1, 0, 1, 1])
```

[40]: # 预测标签概率

```
pred_proba = lr.predict_proba(X_train)
```

```
[41]: pred_proba[:10]
```

```
[41]: array([[0.61849862, 0.38150138],
           [0.13850723, 0.86149277],
           [0.46207721, 0.53792279],
           [0.1916767 , 0.8083233 ],
           [0.86645844, 0.13354156],
           [0.90891601, 0.09108399],
           [0.12699232, 0.87300768],
           [0.90464715, 0.09535285],
           [0.04934066, 0.95065934],
           [0.12648266, 0.87351734]])
```

1.3 3. 模型评估

- 模型评估是为了知道模型的泛化能力。
- 交叉验证 (cross-validation) 是一种评估泛化性能的统计学方法，它比单次划分训练集和测试集的方法更加稳定、全面。
- 在交叉验证中，数据被多次划分，并且需要训练多个模型。
- 最常用的交叉验证是 k 折交叉验证 (k-fold cross-validation)，其中 k 是由用户指定的数字，通常取 5 或 10。
- 准确率 (precision) 度量的是被预测为正例的样本中有多少是真正的正例
- 召回率 (recall) 度量的是正类样本中有多少被预测为正类
- f-分数是准确率与召回率的调和平均

任务一：交叉验证 * 用 10 折交叉验证来评估逻辑回归模型 * 计算交叉验证精度的平均值

交叉验证示意图如下：

提示 4 * 交叉验证在 sklearn 中的模块为 `sklearn.model_selection`

思考 4 * k 折越多的情况下会带来什么样的影响？

```
[42]: from sklearn.model_selection import cross_val_score
```

```
[43]: lr = LogisticRegression(C=100)
      scores = cross_val_score(lr, X_train, y_train, cv=10)
```

/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-


```
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
regression
    extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
    extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
    extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)
```

```
[44]: # k 折交叉验证分数
scores
```

```
[44]: array([0.82089552, 0.7761194 , 0.82089552, 0.79104478, 0.85074627,
          0.86567164, 0.73134328, 0.86567164, 0.75757576, 0.6969697 ])
```

```
[45]: # 平均交叉验证分数
print("Average cross-validation score: {:.2f}".format(scores.mean()))
```

Average cross-validation score: 0.80

混淆矩阵 * 计算二分类问题的混淆矩阵 * 计算精确率、召回率以及 f-分数

提示 5 * 混淆矩阵的方法在 sklearn 中的 sklearn.metrics 模块 * 混淆矩阵需要输入真实标签和

预测标签

思考 5 * 如果自己实现混淆矩阵的时候该注意什么问题

```
[46]: from sklearn.metrics import confusion_matrix
```

```
[47]: # 训练模型
```

```
lr = LogisticRegression(C=100)
lr.fit(X_train, y_train)
```

```
/home/jovyan/.virtualenvs/basenv/lib/python3.7/site-
packages/sklearn/linear_model/_logistic.py:940: ConvergenceWarning: lbfgs failed
to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
extra_warning_msg=_LOGISTIC_SOLVER_CONVERGENCE_MSG)

```
[47]: LogisticRegression(C=100, class_weight=None, dual=False, fit_intercept=True,
        intercept_scaling=1, l1_ratio=None, max_iter=100,
        multi_class='auto', n_jobs=None, penalty='l2',
        random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
        warm_start=False)
```

```
[48]: # 模型预测结果
```

```
pred = lr.predict(X_train)
```

```
[49]: # 混淆矩阵
```

```
confusion_matrix(y_train, pred)
```

```
[49]: array([[354,  58],
        [ 72, 184]])
```

```
[50]: from sklearn.metrics import classification_report
```

```
[51]: # 精确率、召回率以及 f1-score
print(classification_report(y_train, pred))
```

	precision	recall	f1-score	support
0	0.83	0.86	0.84	412
1	0.76	0.72	0.74	256
accuracy			0.81	668
macro avg	0.80	0.79	0.79	668
weighted avg	0.80	0.81	0.80	668

任务三: ROC 曲线 * 绘制 ROC 曲线

提示 6 * ROC 曲线在 sklearn 中的模块为 `sklearn.metrics` * ROC 曲线下所包围的面积越大越好

思考 6 * 对于多分类问题如何绘制 ROC 曲线

```
[52]: from sklearn.metrics import roc_curve
```

```
[53]: fpr, tpr, thresholds = roc_curve(y_test, lr.decision_function(X_test))
plt.plot(fpr, tpr, label="ROC Curve")
plt.xlabel("FPR")
plt.ylabel("TPR (recall)")
# 找到最接近于 0 的阈值
close_zero = np.argmin(np.abs(thresholds))
plt.plot(fpr[close_zero], tpr[close_zero], 'o', markersize=10, label="threshold_
→zero", fillstyle="none", c='k', mew=2)
plt.legend(loc=4)
```

```
[53]: <matplotlib.legend.Legend at 0x7f56cbf16690>
```

